

【专栏二：深度学习04】 - 【一张图讲清楚ReLU vs Sigmoid：为什么ReLU更适合深度网络？】



快乐飞猪ing



· 码龄7年 · 公开 · 编辑



墨衍会员 · AI 创作全网分发

原创 · 2026-03-27 15:14:30 · 618 阅读



AI 写同款 · AI 选题



深度学习 - 23 Whisper LoRA

Dropout(X)这一部分尤其需要和普通数据增强区分开。Base:LoRA:进入 LoRA 分支的 hidden-state activ...

[阅读详情 >](#)

前言

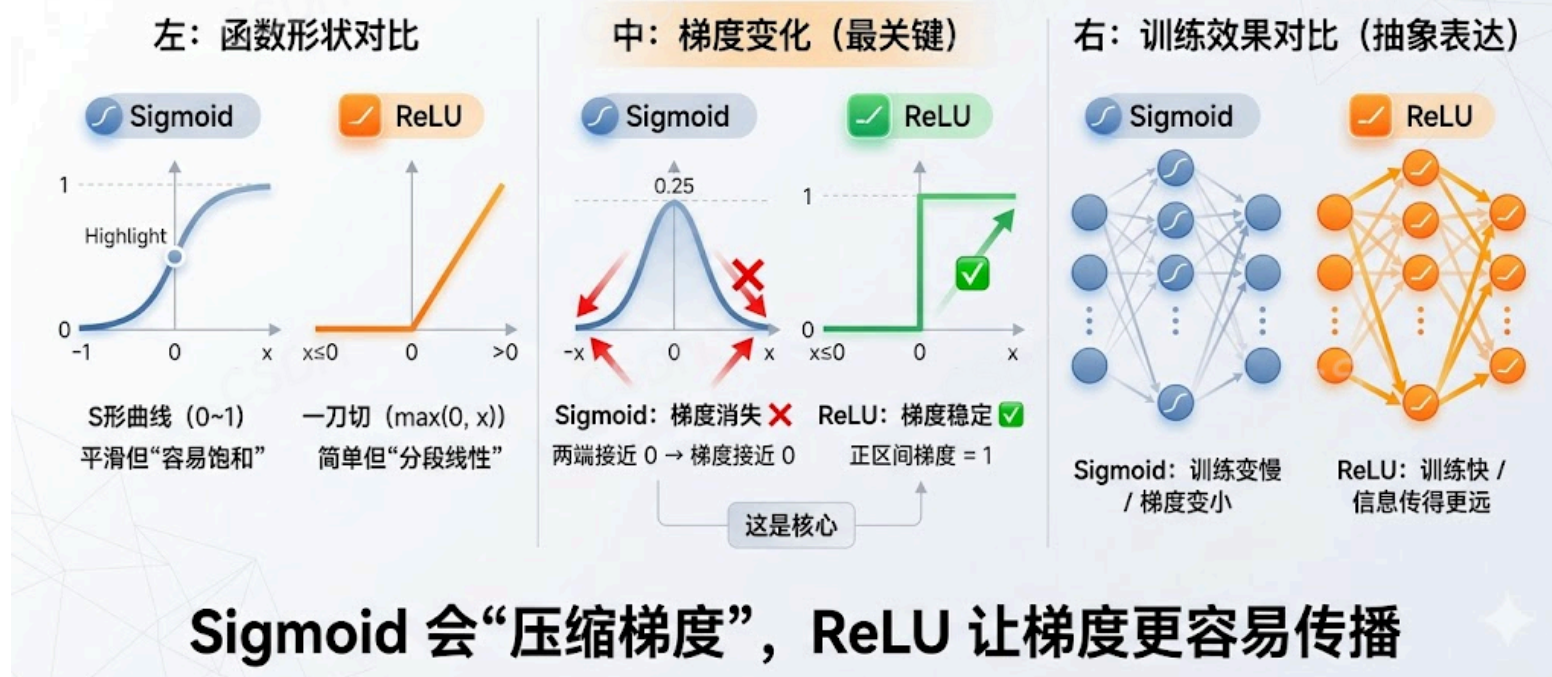
内容来源: csdn.net

作者昵称: 快乐飞猪ing

原文链接: https://blog.csdn.net/m0_46279060/article/details/159539963

作者主页: https://blog.csdn.net/m0_46279060

ReLU vs Sigmoid: 为什么前者更适合深度网络?



Sigmoid 会“压缩梯度”，ReLU 让梯度更容易传播？

我把这句话拆开，尽量讲到你真正能“脑子里有画面”。

这句话是：Sigmoid 会“压缩梯度”，ReLU 让梯度更容易传播。

它其实在讲一件事：神经网络在训练时，后面的“纠错信号”能不能顺利传回前面。

一、先别急着理解梯度，先理解“训练靠什么改参数”

内容来源: csdn.net

作者昵称: 快乐飞猪ing

原文链接: https://blog.csdn.net/m0_46279060/article/details/159539963

作者主页: https://blog.csdn.net/m0_46279060

训练时，**模型**先做一遍**前向传播**，得到**预测结果**。

然后它会发现自己错了多少，再把这个“**错误信息**”往回传。

往回传的目的，是为了告诉前面的参数：

- 你要不要改
- 往哪边改
- 改多少

这个“告诉参数怎么改”的信息，就是**梯度**，所以你可以先把**梯度**理解成：

训练时传递的“纠错信号”

二、什么叫“梯度传播”

假设网络有很多层：

- **第 10 层**
- **第 9 层**
- **第 8 层**
- ...
- **第 1 层**

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159539963

作者主页：https://blog.csdn.net/m0_46279060

模型在最后一层算出误差后，要把这个误差信号一路往前传，才能让前面那些层也跟着学习。

这个过程就叫：**梯度传播**

如果这个信号一路传回去还比较强，前面的层就能学到东西。

如果传着传着几乎没了，前面的层就学不动。

三、现在看 Sigmoid 为什么会“压缩梯度”

先看 Sigmoid:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

它的导数是:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

这个公式你先不用死记，记住一个结论就行：

Sigmoid 的导数最大也只有 0.25，而且大多数时候比 0.25 还小。

这意味着什么？

意味着：**每经过一层 Sigmoid，梯度都会被乘上一个小于 1 的数。**

内容来源: csdn.net

作者昵称: 快乐飞猪ing

原文链接: https://blog.csdn.net/m0_46279060/article/details/159539963

作者主页: https://blog.csdn.net/m0_46279060

比如你可以粗略想象成：

- ****经过一层，乘 0.2 ****
- ****再经过一层，又乘 0.2 ****
- **再经过一层，再乘 0.2**

那最后会变成： $0.2 \times 0.2 \times 0.2 = 0.008$

本来一个“纠错信号”还挺明显，传到前面几层以后，几乎变没了，这就叫：**梯度消失**

所以说：**Sigmoid 会压缩梯度**

这里的“压缩”，意思就是：

梯度每过一层都会被缩小一点，层数一多，前面的层几乎收不到有效信号。

四、为什么 Sigmoid 特别容易这样

因为 Sigmoid 是一条 S 型曲线，它有个特点：

- **中间区域变化比较明显**
- **两头很平**

当输入很大或者很小时，Sigmoid 会接近：

- **0**

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159539963

作者主页：https://blog.csdn.net/m0_46279060

- 或 1

这时候它的斜率会非常小，也就是导数非常接近 0，而反向传播里，梯度就是靠这些导数一层层往回乘的。

所以：

- 如果导数本来就很小
- 一层一层再连乘
- 梯度就会越来越小

这就是为什么 Sigmoid 特别容易造成“梯度传不回去”。

五、ReLU 为什么“让梯度更容易传播”

再看 ReLU：

$$ReLU(x) = \max(0, x)$$

它很简单：

- 输入小于 0 → 输出 0
- 输入大于 0 → 原样保留

它的导数也很简单：

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159539963

作者主页：https://blog.csdn.net/m0_46279060

- $x > 0$ 时, 导数 = 1

- $x < 0$ 时, 导数 = 0

关键在于:

只要输入落在正区间, ReLU 的梯度就是 1。

这是什么意思?

意思是: **梯度经过这一层, 不会被缩小。**

比如后面的梯度是 0.8

经过一层正区间的 ReLU: $0.8 \times 1 = 0.8$

再过一层: $0.8 \times 1 = 0.8$

它不会像 Sigmoid 那样每层都被压缩到越来越小。

所以可以说: **ReLU 让梯度更容易传播**

这里的“更容易传播”, 不是说它会放大梯度, 而是说:

它不会像 Sigmoid 那样, 把梯度一层层缩没掉。

六、你可以把它想象成“水流过管道”

内容来源: csdn.net

作者昵称: 快乐飞猪ing

原文链接: https://blog.csdn.net/m0_46279060/article/details/159539963

作者主页: https://blog.csdn.net/m0_46279060

这个类 Q 比很好懂。

Sigmoid

像一根很细的管子，而且每经过一层都更细一点。

水流过去：

- 第一层少一点
- 第二层再少一点
- 第三层更少
- 到最前面时几乎没水了

这就是“**压缩梯度**”。

ReLU

像一根比较通畅的管子。只要在正区间，水流基本能正常通过，不会每层都被削弱。

所以前面的层还能收到足够的“**纠错信号**”，这就是“**更容易传播**”。

七、再用一个极简数字例子帮你彻底看懂

假设最后传回来的梯度初始值是：1

用 Sigmoid

假设每层平均乘上 0.2

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159539963

作者主页：https://blog.csdn.net/m0_46279060

- 第1层后: $1 \times 0.2 = 0.2$
- 第2层后: $0.2 \times 0.2 = 0.04$
- 第3层后: $0.04 \times 0.2 = 0.008$
- 第4层后: $0.008 \times 0.2 = 0.0016$

你会发现，前面层拿到的梯度已经几乎没了。

用 ReLU

假设每层都在正区间，乘上 1

- 第1层后: $1 \times 1 = 1$
- 第2层后: $1 \times 1 = 1$
- 第3层后: $1 \times 1 = 1$

梯度不会被越传越小，所以前面的层还能学。

八、为什么这会影训练效果

因为训练的本质就是：用梯度去更新参数

如果梯度太小，会发生什么？

- 前面层的参数几乎不更新
- 网络前面的部分学不到东西

内容来源: csdn.net

作者昵称: 快乐飞猪ing

原文链接: https://blog.csdn.net/m0_46279060/article/details/159539963

作者主页: https://blog.csdn.net/m0_46279060

- 训练变慢
- 深层网络很难训练

这就是为什么在深层网络里，Sigmoid 往往不如 ReLU 常用。

九、一句话重新解释这句话

你原来不懂的那句话，我现在给你翻译[🔍]成最直白的版本：

Sigmoid 会让反向传播的梯度一层层变小，传到前面时几乎没了；而 ReLU 在正区间不会缩小梯度，所以误差信号更容易一路传回前面的层。

你可以记成：

Sigmoid 容易把梯度“传没了”，ReLU 更容易把梯度“传回来”。

内容来源：csdn.net

作者昵称：快乐飞猪ing

原文链接：https://blog.csdn.net/m0_46279060/article/details/159539963

作者主页：https://blog.csdn.net/m0_46279060